

Q1. Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

```
mysql> SELECT S.SName, C.ClassTeacher, C.Grade
-> FROM SchoolStudent S
-> JOIN ClassRoom C ON S.ClassID = C.ClassID;
```

| SName  | ClassTeacher | Grade |
|--------|--------------|-------|
| Aditi  | Mrs. Kalpana | 10-A  |
| Bhuvan | Mrs. Kalpana | 10-A  |
| Charan | Mr. Raghavan | 10-B  |
| Deepa  | Mr. Raghavan | 10-B  |
| Elango | Mrs. Shanthi | 9-A   |

5 rows in set (0.005 sec)

Q2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

```
mysql> SELECT D.DoctorID, D.DName, COUNT(A.ApptID) AS TotalAppointments
-> FROM Doctor D
-> JOIN Appointment A ON D.DoctorID = A.DoctorID
-> WHERE MONTH(A.ApptDate) = MONTH(CURDATE())
-> OR MONTH(A.ApptDate) = 1
-> GROUP BY D.DoctorID, D.DName
-> HAVING COUNT(A.ApptID) > 10;
```

| DoctorID | DName      | TotalAppointments |
|----------|------------|-------------------|
| 1        | Dr. Sundar | 11                |

1 row in set (0.001 sec)

Q3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

```
mysql> SELECT ProductName
-> FROM Product
-> WHERE ProductID NOT IN (SELECT ProductID FROM CustOrder);
+-----+
| ProductName |
+-----+
| Notebook Set |
| Desk Lamp   |
+-----+
2 rows in set (0.023 sec)
```

Q4. For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

**Student, Course, and Enrollment relations:**

$DBMS\_Students = \pi_{SID} ( \sigma_{CourseName='DBMS'} ( Enrollment \bowtie Course ) )$

$OS\_Students = \pi_{SID} ( \sigma_{CourseName='OS'} ( Enrollment \bowtie Course ) )$

**Result** =  $\pi_{Name} ( (DBMS\_Students \cap OS\_Students) \bowtie Student )$

**intersection ( $\cap$ )** — the set of student IDs that appear in both the DBMS list and the OS list then joins back to Student to get names.

Q5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

```
mysql> CREATE OR REPLACE VIEW BelowAvgSalaryView AS
-> SELECT P1.EmpID, P1.EName, P1.DeptID, P1.Salary
-> FROM PayrollEmployee P1
-> WHERE P1.Salary < (
->     SELECT AVG(P2.Salary)
->     FROM PayrollEmployee P2
->     WHERE P2.DeptID = P1.DeptID
-> );
```

Query OK, 0 rows affected (0.079 sec)

```
mysql> -- View the employees below their department average
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT * FROM BelowAvgSalaryView;
+-----+-----+-----+-----+
| EmpID | EName   | DeptID | Salary |
+-----+-----+-----+-----+
| 1     | Kavya   | 1      | 45000.00 |
| 3     | Mahesh  | 2      | 38000.00 |
| 6     | Pavithra | 3      | 39000.00 |
+-----+-----+-----+-----+
3 rows in set (0.011 sec)
```

```
mysql> -- Update their salary by 10%
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE PayrollEmployee
-> SET Salary = Salary * 1.10
-> WHERE EmpID IN (
->     SELECT EmpID FROM (
->         SELECT EmpID FROM BelowAvgSalaryView
->     ) AS temp
-> );
Query OK, 3 rows affected (0.019 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> -- Verify the update
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT * FROM PayrollEmployee;
+-----+-----+-----+-----+
| EmpID | EName      | DeptID | Salary  |
+-----+-----+-----+-----+
| 1     | Kavya      | 1      | 49500.00 |
| 2     | Lokesh     | 1      | 60000.00 |
| 3     | Mahesh     | 2      | 41800.00 |
| 4     | Nandhini   | 2      | 52000.00 |
| 5     | Om Prakash | 3      | 71000.00 |
| 6     | Pavithra   | 3      | 42900.00 |
+-----+-----+-----+-----+
6 rows in set (0.000 sec)
```

Q6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

```
mysql> SELECT C.CName, A.AccountID, A.Balance, Br.BranchName, Br.City,
->     T.TxnType, T.Amount, T.TxnDate
-> FROM Customer C
-> JOIN Account A ON C.CustomerID = A.CustomerID
-> JOIN Branch Br ON A.BranchID = Br.BranchID
-> JOIN BankTransaction T ON A.AccountID = T.AccountID
-> ORDER BY C.CName, T.TxnDate;
+-----+-----+-----+-----+-----+-----+-----+-----+
| CName | AccountID | Balance | BranchName | City | TxnType | Amount | TxnDate |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Raghav | 101 | 25000.00 | Anna Nagar | Chennai | Deposit | 5000.00 | 2025-01-01 |
| Raghav | 101 | 25000.00 | Anna Nagar | Chennai | Withdraw | 2000.00 | 2025-01-02 |
| Sathya | 102 | 42000.00 | Anna Nagar | Chennai | Deposit | 10000.00 | 2025-01-03 |
| Tarun | 103 | 15000.00 | T Nagar | Chennai | Deposit | 3000.00 | 2025-01-04 |
+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.031 sec)
```

Q7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

```
mysql> SELECT Category, SUM(Amount) AS TotalSales
-> FROM Sales
-> GROUP BY Category
-> HAVING SUM(Amount) > 10000;
+-----+-----+
| Category          | TotalSales |
+-----+-----+
| Electronics       | 65400.00  |
| Home Appliances   | 37600.00  |
+-----+-----+
2 rows in set (0.009 sec)
```

Q8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

$$\{ m.MName \mid m \in \text{Member} \wedge \\ \forall g \in \text{Genre} ( \exists b \in \text{Book} \exists br \in \text{Borrow} ( \\ b.GenreID = g.GenreID \wedge \\ br.BookID = b.BookID \wedge \\ br.MemberID = m.MemberID \\ )) \}$$

Q9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

```
mysql> -- Valid insert (passes both constraints)
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO InventoryItem (ItemID, ItemName, Quantity, Price, SupplierID)
-> VALUES (4, 'Wire Coil', 200, 150, 1);
Query OK, 1 row affected (0.013 sec)

mysql> -- Violates CHECK constraint: negative quantity (should error)
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO InventoryItem (ItemID, ItemName, Quantity, Price, SupplierID)
-> VALUES (5, 'Broken Item', -10, 100, 1);
ERROR 3819 (HY000): Check constraint 'InventoryItem_chk_1' is violated.
mysql> -- Violates CHECK constraint: price must be > 0 (should error)
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO InventoryItem (ItemID, ItemName, Quantity, Price, SupplierID)
-> VALUES (6, 'Free Item', 10, 0, 1);
ERROR 3819 (HY000): Check constraint 'InventoryItem_chk_2' is violated.
mysql> INSERT INTO InventoryItem (ItemID, ItemName, Quantity, Price, SupplierID)
-> VALUES (7, 'Ghost Item', 10, 100, 99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`dbms_lab`.`InventoryItem`, CONSTRAINT `fk_inventoryitem_supplier` FOREIGN KEY (`SupplierID`) REFERENCES `Supplier` (`SupplierID`))
```

```
mysql> -- Verify only the valid row was added
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT * FROM InventoryItem;
```

```
+-----+-----+-----+-----+-----+
| ItemID | ItemName      | Quantity | Price  | SupplierID |
+-----+-----+-----+-----+-----+
|      1 | Bolt Pack     |      500 | 50.00  |           1 |
|      2 | Steel Sheet   |      120 | 750.00 |           1 |
|      3 | Paint Bucket  |       80 | 600.00 |           2 |
|      4 | Wire Coil     |      200 | 150.00 |           1 |
+-----+-----+-----+-----+-----+
4 rows in set (0.007 sec)
```

Q10. Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

#### Materialized Attendance Summary:

```
mysql> DROP TABLE IF EXISTS MonthlyAttendanceSummary;
Query OK, 0 rows affected, 1 warning (0.010 sec)
```

```
mysql> CREATE TABLE MonthlyAttendanceSummary AS
-> SELECT H.EmpID, H.ENAME,
->        DATE_FORMAT(A.AttDate, '%Y-%m') AS Month,
->        SUM(CASE WHEN A.Status = 'Present' THEN 1 ELSE 0 END) AS DaysPresent,
->        SUM(CASE WHEN A.Status = 'Absent' THEN 1 ELSE 0 END) AS DaysAbsent
-> FROM HREmployee H
-> JOIN Attendance A ON H.EmpID = A.EmpID
-> GROUP BY H.EmpID, H.ENAME, DATE_FORMAT(A.AttDate, '%Y-%m');
Query OK, 2 rows affected (0.053 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM MonthlyAttendanceSummary;
```

```
+-----+-----+-----+-----+-----+
| EmpID | EName  | Month   | DaysPresent | DaysAbsent |
+-----+-----+-----+-----+-----+
|      1 | Yazhini | 2025-01 |           2 |           1 |
|      2 | Zafar   | 2025-01 |           2 |           0 |
+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

#### Materialized Performance Summary:

```
mysql> CREATE TABLE MonthlyPerformanceSummary AS
-> SELECT H.EmpID, H.ENAME, P.Month, P.Score
-> FROM HREmployee H
-> JOIN Performance P ON H.EmpID = P.EmpID;
Query OK, 2 rows affected (0.023 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM MonthlyPerformanceSummary;
```

```
+-----+-----+-----+-----+
| EmpID | EName  | Month   | Score |
+-----+-----+-----+-----+
|      1 | Yazhini | January |     85 |
|      2 | Zafar   | January |     90 |
+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```